Press release

SHOGI launches authority infrastructure that prevents unauthorized AI-agent actions.

SHOGI's cryptographic proof of authority controls whether protected agent actions execute, combining post-quantum identity-based encryption, quorum approval, and deterministic execution.

Palo Alto, December 1st, 2027 — SHOGI Systems, Inc. today announced general availability of SHOGI, an **authorization** infrastructure for AI agents, connected devices, and autonomous systems operating in mission-critical environments. Within SHOGI's protected execution boundary, a critical action can execute only after its cryptographic proof of authority has been validated and an independent quorum of agents has approved that exact action, with additional human approval required for higher-risk actions. This makes verified authority a prerequisite for execution, rather than relying on an agent to follow instructions about what it may do.

Gartner forecasts that 33% of enterprise software applications will include agentic AI by 2028, up from under 1% in 2024, and that at least 15% of day-to-day work decisions will be made autonomously by then. The same research predicts that more than 40% of agentic AI projects will be canceled by the end of 2027, citing inadequate risk controls among the reasons. Monitoring and logging provide visibility into agent activity, but visibility alone does not establish whether an agent has valid authority to execute a consequential action. An agent that is tricked, misconfigured, or compromised can misuse its existing permissions at machine speed and across systems.

SHOGI makes verified authority a prerequisite for action. Every agent has a cryptographic identity bound to its human creator and group, its assigned objective, and its authority boundaries. Critical actions require approval by a quorum of independent agents, which verifies the required evidence before approving the exact action. Higher-risk actions also require human approval. Routine actions run within the identity's limits without waiting for approval. The resulting approval is single-use and bound to that action, so it cannot authorize a different action or be reused. Within SHOGI's protected execution boundary, if any required check or approval is missing or invalid, the action does not run.

Every SHOGI deployment begins with mission-assurance analysis using System-Theoretic Process Analysis for Security (STPA-Sec), developed by SHOGI advisor and STPA-Sec Integration Lead Dr. William "Dollar" Young as the security extension of MIT Professor Nancy Leveson's STPA methodology. The analysis finds which actions in a workflow would be unsafe, and under what conditions, and turns the losses a customer cannot accept into concrete rules before anything goes live. Those rules become Ordered Identities (patent pending): each agent's role, mission, authority tier, jurisdiction, allowed actions, and time window, written in a fixed order. The identity serves as the public key, and the corresponding private key is derived from it. The identity is the policy, so an agent tricked into trying something outside its authority holds no key that would enable it.
Those keys come from SHOGI's Post-Quantum Identity-Based Encryption.

With Identity-Based Encryption, the identity itself acts as the public key, so there are no separate certificates or permission lookups for an attacker to tamper with. SHOGI's lattice-based version, combined with signcryption that signs and encrypts each request in one step, is designed to resist attacks from future quantum computers. Requests cannot be forged or altered, and approvals and audit

records remain trustworthy. Keys are issued by the Identity Authority (PKG) from secure hardware, with its master secret split among several key authorities, so no single insider can grant additional authority.

At run time, every request goes to the authorization gate. Each request is rated for risk, and critical actions go to a quorum of independent agents, with additional human approval required for higher-risk actions. Approvers must verify the required evidence before voting, so separation of duties is enforced by cryptography rather than a setting one person or agent can change. The authorization gate issues a single-use cryptographic approval only after all required checks and approvals are complete. That approval authorizes only the exact action requested. On each protected system, a designated driver, a deterministic component that runs no AI and holds the real credentials, performs the exact action only with a valid, matching approval and signs a receipt; any action without a valid, matching approval is refused. If any required evidence, vote, or verification is missing, the gate denies the request. SHOGI works alongside existing access systems. Additionally, every request, vote, approval, refusal, revocation, and key recovery is written to an audit record, giving auditors evidence that controls held, including for attempts that were blocked in real time.

"SHOGI's vision is simple. No agent should be able to take a consequential action unless it can prove it has the authority to do so, and that proof should hold up even against tomorrow's quantum computers. SHOGI requires agents to prove their ***authority*** *before taking consequential actions, giving organizations the confidence to deploy them in mission-critical workflows."* — Bill Montgomery, CEO, SHOGI Systems, Inc.

"Our agents prepare thousands of payments a day. One scenario that kept us up at night was an agent being talked into changing a supplier's bank details and then marking its own callback as done. With SHOGI, an agent that proposes a change simply cannot verify it, and the payment system is guaranteed to not release funds to an account the quorum never approved. We moved from catching fraud after the money left to stopping it before it can." — Ms. Money Maker, Head of Treasury Operations, Bank

"We let agents build, test, and promote releases, which is great until one of them ships something nobody approved. SHOGI ties every production release to a single-use approval from our release quorum, with test evidence checked before anyone votes. An agent can still do ninety percent of the work. It just cannot push to production on its own say-so. Since we implemented SHOGI, we have been able to eliminate 100% of unauthorized releases" — Mr. Software Release, Vice President of Release Engineering, ISV

To start using SHOGI visit SHOGIsystems.com.

FAQ

Who is SHOGI for? SHOGI is for organizations that need proof that an AI agent, device, or autonomous system is authorized to act. Use cases span enterprise AI, financial operations, software builds and releases, operational technology (OT), the Internet of Things (IoT), and physical AI, including robotics and autonomous machines. SHOGI is especially relevant where unauthorized actions could cause significant financial loss, disrupt critical operations, or endanger people.

How is this different from monitoring and logging tools? Monitoring and logging provide visibility into activity as it happens and after the fact. SHOGI requires valid proof of authority before a protected critical action can execute. The two are complementary: keep monitoring for visibility and add SHOGI for provable authority at the point of action.

Do I have to replace my existing access and identity systems? No. Existing systems still control how an agent reaches a service. SHOGI controls whether a specific action is authorized. For protected actions, the target system, or a SHOGI checkpoint in front of it, only acts on a valid approval for that exact action. Agents no longer keep their own credentials for those actions.

What happens if an approver or voting seat is unavailable? An unavailable approver does not stop the action if the required quorum and any mandatory human approvals can still be obtained. If those requirements cannot be met, the action waits. SHOGI fails closed.

Can a compromised agent still cause harm? Only within the scope of its own identity, or in systems not wired to SHOGI. Anything outside its job, or above the risk threshold without a quorum, is refused and logged.

Why post-quantum cryptography now? Cryptographic systems deployed today must be quantum-ready. Approvals and audit records may need to remain trustworthy for years, so protection against future quantum attacks must be designed in from the start. SHOGI's Post-Quantum Identity-Based Encryption uses lattice-based cryptography designed to resist those attacks.

Do I need to buy security hardware? No. SHOGI hosts the key infrastructure in its own secure hardware. Customers who want full control of a closed system can use their own hardware security modules (HSMs).

Can customers implement SHOGI themselves? For a financial institution, wiring the authorization gate means changing payment, release, and record systems so they refuse unapproved actions, work many customers may want help with. SHOGI has partnered with several system integrators serving multiple industries including banking and software-delivery practices, and security consultancies with STPA-Sec experience to support customers in their implementation if needed.

What does SHOGI not protect? Anything not wired through the gate. A direct database edit or bulk import outside the governed path stays open. SHOGI also verifies authority, not correctness: a properly authorized quorum can still approve a bad change.

Who else is working on authorization for AI agents? Many strong companies, and most of them handle a part of the problem SHOGI depends on. Identity providers and token services such as Okta/Auth0, Keycard, and Descope establish who an agent is and whom it acts for. Policy engines such as Cerbos, AuthZed (SpiceDB), and AWS Bedrock AgentCore Policy decide whether a request fits the rules. Gateways such as Arcade.dev, Permit.io, and Pomerium can check tool calls and keep upstream credentials away from agents. Tigera Lynx governs agent fleets through workload identity and policy

enforcement. Secure runtimes and monitoring tools such as NVIDIA OpenShell with Claude Managed Agents, Rein Security, and Kontext Security contain agents and watch their behavior. SHOGI is designed to work with all of them.

How does SHOGI work with these companies? SHOGI does not replace identity providers, policy engines, gateways, or secure runtimes; it builds on them. An identity provider such as Okta or Keycard can say who an agent is. A policy engine such as Cerbos, Cedar, or SpiceDB can evaluate the rules. A runtime such as OpenShell can contain the agent. A gateway such as Arcade or Permit.io, which sits where agents act, can require a SHOGI approval before it runs a high-consequence call. SHOGI adds the final step: a single-use approval for one exact action, checked by the component that performs it, followed by a signed receipt that an auditor can verify without relying on SHOGI's word. A designated driver, a deterministic component that runs no AI and holds the real credentials, verifies the permit, required approvals, time window, and single-use status before performing the exact operation and signing a receipt. Customers keep the tools they already use and add proof of authority wherever an unauthorized action would be costly.

Is SHOGI competing with NVIDIA OpenShell and Claude Managed Agents? No. The two are complementary. NVIDIA and Anthropic provide a controlled environment for agents to run in: Anthropic keeps the agent's credentials in a vault the agent never sees, and NVIDIA's open-source OpenShell runtime checks every action against a deny-by-default policy and logs what it allows or blocks. SHOGI binds mission constraints, quorum and human approval, and execution into each protected action. OpenShell is a natural integration path for SHOGI.

What does SHOGI add that is not already available? Checking actions before they run, keeping credentials away from agents, and pausing for one person's approval are now common, and several of the companies above do all three well. SHOGI adds four things. Our review found no alternative publicly documented as delivering this complete combination in production: approval of an action by several independent parties, in a form the executing system can check; a record of each approved action that an outside auditor can verify without trusting the vendor; authority bound into the agent's key, so an agent holds no key for actions outside its job; and lattice-based cryptography protecting authority itself, designed to resist future quantum attacks. Together, these let an organization prove that a specific action was authorized, at the point where it takes place.